QuantRule — Architecture Characteristics

QuantRule

2026-11-11

Contents

1 QuantRule — Architecture Characteristics **1**
1.1 1. Method . . . 1
1.2 2. Derivation . . . 1
1.3 3. Driving characteristics . . . 1
1.4 4. Supporting characteristics . . . 2
1.5 5. Deliberately not optimized . . . 2
1.6 6. Style and quantum . . . 3
1.7 7. Trade-offs . . . 3
1.8 8. Future trigger . . . 3
1.9 9. Evidence . . . 3

1 QuantRule — Architecture Characteristics

1.1 1. Method

The characteristics are derived from the kata's demands and kept short: three drivers, one enabling characteristic, and the supporting properties that protect the product promise. Each characteristic has a measurable target and current evidence. The style and quantum are recorded after the table.

1.2 2. Derivation

Kata demand	Characteristic
Same input produces the same evidence	Reproducibility
No look-ahead; events only when knowable	Temporal integrity
Evidence can be inspected and defended	Auditability
Every reading and rule is testable	Testability (enabling)
One local pipeline with clear boundaries	Modularity
A Linux installer a researcher can run	Deployability
Contracts can evolve without rewriting the engine	Evolvability
No cloud bill and no data egress	Cost and local security

1.3 3. Driving characteristics

Characteristic	Target	Measurement	Current score (1–5)	Evidence
Reproducibility	A fixed fixture run is byte-identical across repeated runs	Rerun comparison, golden fixtures	5	Determinism and byte-identical rerun tests
Temporal integrity	No output at bar t changes when later bars are appended	Causality and prefix tests	5	Model, event, and strategy boundary tests
Auditability	Every evidence bundle names data version, config hash, code commit, artifacts	Provenance schema check and export review	5	Model/stage provenance, versioned run summaries, and research memory
Testability (enabling)	Fast suite plus desktop E2E and installer smoke	`cargo test`, UI E2E, install test	5	1,476 Rust tests, 7 frontend unit tests, 1 Playwright E2E, 4 desktop backend tests, clean install

1.4 4. Supporting characteristics

Characteristic	Target	Measurement	Current score (1–5)
Modularity	One crate per pipeline stage plus a shared application service; no dependency cycles	Crate graph and structural checker	5
Deployability	Clean Linux install in under five minutes	Installer smoke test	4
Evolvability	Versioned configs and research-memory schema	Contract and migration tests	4
Cost	No cloud bill; static download site	Hosting and infrastructure inventory	5
Local security	No engine network calls; data stays local	Offline test and network isolation	5

1.5 5. Deliberately not optimized

Characteristic	Decision	Consequence
Scalability	Single user, single machine	No horizontal scaling
Elasticity	Batch runs, no service	No autoscaling

Characteristic	Decision	Consequence
Availability	Local app, no uptime target	A failed run is an error, not an outage
Multi-tenancy	No accounts or team workspace	Team cloud is future work
Interoperability	File contracts only	No API or plugin marketplace

1.6 6. Style and quantum

- **Style:** pipes-and-filters inside a modular monolith.
- **Quantum:** one local deployable app with local files and SQLite; the CLI and the Tauri UI are adapters over the same application service.
- **Distribution:** a static download page for Linux installers.
- **Future:** a team control plane would be a second quantum; it is not part of this increment.

1.7 7. Trade-offs

Trade-off	Decision	Consequence
Local-first vs collaboration	Keep data and compute local	No team sync in v1
Reproducibility vs change velocity	Golden fixtures and versioned contracts	Intentional changes are deliberate
Simplicity vs future cloud	Defer the control plane	A future ADR and characteristics review are required
Linux-only vs reach	Ship Linux first	Windows and macOS are future work

1.8 8. Future trigger

If a collaborative team cloud is introduced, re-derive the characteristics: security, multi-tenancy, sync, availability, and cost will need new targets. They are not current drivers.

1.9 9. Evidence

- `cargo test --workspace`: 1,476 passed, 0 failed, 8 ignored.
- Desktop: `npm run check` clean; 7 unit tests; 1 Playwright E2E; 4 backend tests.
- Linux: `.deb` and `.AppImage` build; the deb installs in Ubuntu 24.04 and launches under Xvfb; the AppImage launches with the network namespace removed.
- Determinism: repeated model bundles and `run check` output are byte-identical.
- Research memory: 22 tests cover schema, migrations, CRUD, export/import, and backup/restore.
- Repository checks: structure, file size, links, terms, command surface, workflows, changelog, releases, registry, validation, and verification.
- The Phase 3 verification report is `docs/reports/2026-11-11/implementation-verification.md`.

The scores are course evidence, not a claim of universal quality. Deployability is 4 because the clean install is containerized and no bare-metal install has been run; local security is 5 for the current scope because the engines have no network path and the desktop app launches offline.